

CMPSC 463 Final Project:

Emergency Aid

Distribution Optimizer

Team Members:

John Francis Teetz, Mekhi Saunders

Due Date: April 26th, 2026

Project Overview

Objective - The goal of this project will be to implement a decision-based tool that will optimize the distribution of emergency supplies, such as food, water, or medicine, to zones that have been affected by armed conflict (war). Many severe limitations arise from war and conflict, such as limited delivery vehicles, low vehicle capacity, and optimal delivery time windows (ceasefires), road hazards (blocked roads/zones), lack of information, and time sensitivity. This system aims to optimize and output a delivery plan for these zones in urgent need of assistance, maximizing the urgency of the need for aid while taking into account all of the constraints. Our final project includes two selection algorithms (greedy and priority queue), scenarios based on real areas, fog-of-war (uncertainty/lack of info), and an interactive web interface.

System Functionality (How the System will work) -

- **Phase 1 - Zone Selection/Urgency Scoring**
 - Each affected zone will be given a priority score ($\text{urgency}/(\text{distance} \times \text{risk})$)
 - **Greedy Selection algorithm:** Trucks will be loaded one at a time based on the zone priority levels. Zones will keep getting added to the aid delivery list until no more zones are available or the trucks are all loaded.
 - **Priority Queue algorithm:** uses a max-heap in order to assign zones to trucks with the largest remaining capacity that will be able to fit the zone's needs.
 - Algorithms ignore zones blocked by users or that are inaccessible.
 - Results in a set of zones that will receive aid, prioritized by how critical the situation and need for aid are, as well as the accessibility of each zone.
- **Phase 2 - Delivery Optimization Routing w/ Nearest Neighbor**
 - After the zones have been selected for delivery, the routes of the trucks need to be optimized for the travel/delivery time to be within the ceasefire window
 - **Nearest neighbor algorithm:**
 - The algorithm will start at the supply depot, and the algorithm chooses the next unvisited zone that is closest to the current location
 - The truck will return to the depot after the last zone delivery has been completed

- The total optimal route time will be calculated in the algorithm, then converted to km; time is calculated at 50km/hr
 - Routes are drawn as colored lines on a map for visualization purposes
- **Phase 3 - Interactive Web Interface**
 - This UI uses a Flask backend to showcase a Leaflet map with real geographic coordinates, a left panel for scenario selection, a right panel for metrics of the optimizer, as well as a map legend footer and system header
 - .Extra features: fog of war (uncertain zones with a probability of being accessible/non-accessible), advanced time (unserved zones urgency increases every hour), reveal fog, and block roads (block/unblock).

User Experience - The Emergency Aid Distribution optimizer provides a clean dashboard with three main components combined with a header and footer. This UI is designed for mobile and laptop use for on-field use in these serious situations. The main components are as follows:

- Left Panel for Configuring the Optimizer -
 - Scenario selector with four real-world scenarios (Aleppo Siege, Idlib Displacement, Bekaa Valley Flood, and Mosul Post-Conflict)
 - Sliders for selecting the constraints: number of trucks (1-8) and truck capacity (5k-30k)
 - Time simulation: slider for hours per tick, time elapsed, and buttons to advance/reset time simulation
 - Zone list when optimizer is ran: list of all the zones (including fog-of-war zones), click on a zone to block/unblock it
- Map in the Center of the Optimizer Website -
 - Leaflet map with real coordinates, full map functionality including zooming in/out and clicking the zone/truck/depot markers for more info
 - In the Algorithms tab above the map, you can use to switch between (greedy, priority queue, side by side)
 - All of the zone markers that are color-coded by the intensity of their urgency for aid: blue = 1 - 4; orange = 5 - 7; red = 8-10
 - Fog of war icons that can be revealed

- Colored lines of the routes after the aid optimization are performed
- Right Panel for Metrics/Activity -
 - Comparison of greedy vs. priority queue algorithms using two separate columns with the listed metrics: coverage percentage, aid delivered, urgency sum, and total delivery runtime. Below the comparison columns will be a summary of which algorithm performed better.
 - Supply breakdown of aid (medicine, food, shelter materials) within each truck
- Header - System Title (Emergency Aid Optimizer abbreviated as 'EAOPT') with status tags of the current conflict status, online aid depots, and current zone queue
- Footer - Colour Legend of the map and version tag of the current Emergency aid optimizer

Significance - In a lot of situations with armed conflicts, like in Ukraine, Gaza, etc., companies face these exact problems of limited trucks/transport, hazardous roads, and short windows of opportunity to send aid. This algorithm aims to solve that by providing an algorithm that will get supplies to the most crucial zones/areas, which hopefully can help save lives. Through this project, both of us will be able to showcase all of our learning within this course to solve a problem through algorithmic design and showcase it through the web. This algorithm will be simple and effective for people to use on the go through mobile, which will help a ton for urgent decision-making in real situations where this tool is needed. People will be able to run this Aid Optimizer from mobile phones or laptops on the go without the need for internet access or wifi, which will help in real-world situations.

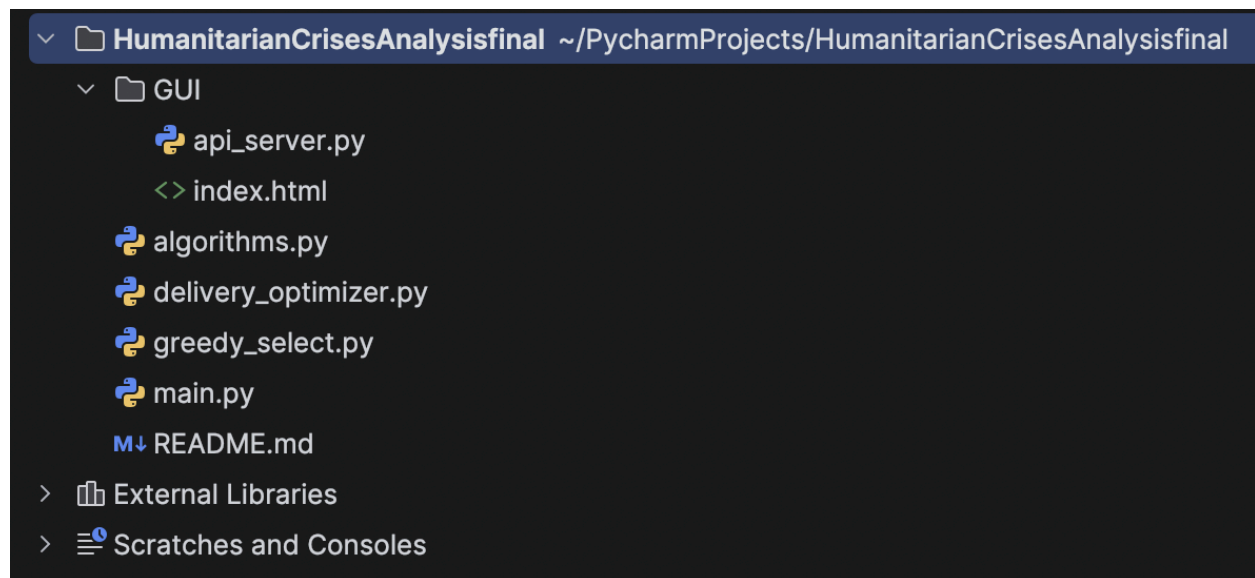
Code Structure

The Emergency Aid Distribution optimizer will be organized by the following files :

- **index.html**: Front end with all the UI functionalities, written in JavaScript
- **api_server.py**: Flask backend, which allows index.html to function, provides the necessary endpoints for the frontend
- **greedy_select.py**: Contains the greedy_selection() algorithm which takes the given inputs and computes a priority score
- **delivery_optimizer.py**: contains nearest neighbour routing and conversion of degrees to km

- **algorithms.py**: contains priority queue selection, urgency decay, and fog of war algorithms
- **main.py**: Used to demo the algorithms before GUI usage
- **README.md**: Documentation of all files and the usage instructions

Here is a screenshot of the exact code structure of our project:



Algorithm Descriptions

Greedy Selection (greedy_select.py) - This algorithm assigns each zone a priority score, calculated by $\text{urgency}/(\text{distance} \times \text{risk})$, and sorts each zone in descending order. Based on these scores, each truck will be filled one at a time to trucks with enough capacity remaining. Returns the selected zones, truck assignments, total kilograms of aid, and total urgency score.

Priority Queue Selection (algorithms.py)- This algorithm uses the same priority score calculation formula in order to utilize it with a max heap. The maxheap pops the highest priority zones and then assigns them to trucks with the largest capacity remaining that will fit the zones' demands of aid. This means that the truck assigned to the zones will not always be the first truck in the order, which helps balance the load across the fleet of trucks. This algorithm can lead to better coverage or a different set of zones receiving aid in some instances, as compared to the greedy selection algorithm.

Nearest Neighbour Routing (delivery_optimizer.py) - This algorithm will be used by both of the previous algorithms discussed to plan the routes of the delivery trucks. Trucks will start at the depot, then go to the closest zone that has not yet been visited. After they reach the last zone or run out of aid, the trucks will return to their depot. Trucks are set to travel at 50 km/hr, and the time elapsed will be calculated as total km / 50 km.

Urgency Decay (algorithms.py) - When the user of the software UI clicks on the ‘advance time’ button, every zone that has still not received their aid will gain an increased amount of urgency (calculated by $0.3 * \text{hours_elapsed}$), which is capped at 10. This functions as a tool to showcase the increasingly worse conditions that the zone is currently experiencing due to not receiving their aid yet.

Fog of War (algorithms.py) - Some zones will have limited information regarding them, and will be displayed as hidden on the leaflet map. When the user clicks on the ‘reveal fog zones’ button in the UI, there is a random chance for the fog zone to either be accessible (.65) or inaccessible (.35). If a zone is determined to be inaccessible, then it will be blocked from the software and excluded from the subsequent optimization runs.

Verification of Algorithms

In order to verify the functionality of each algorithm, we used toy examples as well as testing within the main.py program file. The outputs of both performed correctly for all algorithms (greedy select, priority queue, urgency decay, fog of war, and nearest neighbor routing) as shown in the screenshots below:

Greedy Selection: We ran the greedy selection algorithm on a set of five zones A through E, with a depot at (0,0). The trucks in this scenario all had a capacity of 18k kg with an aid demand of 51k kg. The algorithm computed a priority score for each zone based on their parameters, and this resulted in a list of zones sorted by their priority score:

Emergency Aid Distribution Optimizer -

Affected zones Sorted by Priority:

Zone	Demand (kg)	Urgency (1-10)	Risk (1-5)	Distance (km)	Priority Score
A	12000	10	5	157.0	1.414
E	10000	9	5	157.0	1.273
B	8000	8	4	314.0	0.707
C	15000	7	3	470.9	0.550
D	6000	6	4	627.9	0.265

The greedy algorithm filled trucks in order, depending on the remaining capacity they had:

```
Truck Zone Assignment reasoning

Truck 1:
- Zone A: priority score 1.414, demand 12000 kg
  Fit within remaining capacity (18000 kg)
- Zone D: priority score 0.265, demand 6000 kg
  Fit within remaining capacity (6000 kg)
  Remaining capacity after loading: 0 kg

Truck 2:
- Zone E: priority score 1.273, demand 10000 kg
  Fit within remaining capacity (18000 kg)
- Zone B: priority score 0.707, demand 8000 kg
  Fit within remaining capacity (8000 kg)
  Remaining capacity after loading: 0 kg

Truck 3:
- Zone C: priority score 0.550, demand 15000 kg
  Fit within remaining capacity (18000 kg)
  Remaining capacity after loading: 3000 kg
```

All of the five zones within the toy example received their aid, and the optimizer achieved:

- 100% zone coverage
- Total Aid Delivered = 51,000 kg
- Total delivery time elapsed = 62.8 hours
- Total urgency sum = 40 (calculated by the sum of all urgency scores from zone A-E).

This algorithm output matches the manual verification we performed for this test example:

```

Truck Route details -

Truck 1: assigned zones ['A', 'D']
Route order: start at depot
  1. Go to Zone A - 157.0 km, arrival total 157.0 km
  2. Go to Zone D - 470.9 km, arrival total 627.9 km
  Then return to depot - 627.9 km, total route 1255.8 km
Total travel distance: 1255.8 km, estimated time: 25.1 hours (at 50 km/h)

Truck 2: assigned zones ['E', 'B']
Route order: start at depot
  1. Go to Zone E - 157.0 km, arrival total 157.0 km
  2. Go to Zone B - 470.9 km, arrival total 627.9 km
  Then return to depot - 314.0 km, total route 941.9 km
Total travel distance: 941.9 km, estimated time: 18.8 hours (at 50 km/h)

Truck 3: assigned zones ['C']
Route order: start at depot
  1. Go to Zone C - 470.9 km, arrival total 470.9 km
  Then return to depot - 470.9 km, total route 941.9 km
Total travel distance: 941.9 km, estimated time: 18.8 hours (at 50 km/h)

Summary
Total aid delivered: 51000 kg (51.0 metric tons)
Requested aid amount: 51000 kg
Coverage: 100.0% of requested aid
Total urgency sum (sum of urgencies of delivered zones): 40
Total delivery time for all trucks (combined): 62.8 hours

```

Priority Queue Selection: The same testing example was used for the priority queue algorithm. This algorithm, instead of filling zones sequentially, would pop the highest priority score zones and assign them to specific trucks with the largest remaining capacity that could meet their needs. This algorithm produced the zone assignments of:

- Truck 1 - A
- Truck 2 - E
- Truck 3 - B and D

Zone C was the only zone that wasn't served in this example because no truck had enough remaining capacity to fit its needs after the higher priority zones were assigned. This resulted in:

- Total aid delivered = 36,000 kg
- Urgency sum = 33
- Coverage = 70.6%
- Total delivery time elapsed = 37.7 hours

The following screenshot showcases both the results from the priority queue algorithm and the comparison between this algorithm and greedy selection:


```
Priority Queue Selection on full dataset (5 zones, 3 trucks, 18000 kg each)
Priority Queue assignments: [['A'], ['E'], ['B', 'D']]
Total kg delivered: 36000 Total urgency: 33
Selected zone IDs: ['A', 'B', 'D', 'E']

Algorithm comparison (same toy example):
Greedy Algorithm - Zones served: 5, Total kg: 51000, Urgency sum: 40, Coverage: 100.0%, Total hours: 62.8
Priority Queue - Zones served: 4, Total kg: 36000, Urgency sum: 33, Coverage: 70.6%, Total hours: 37.7
```

The results showcase how greedy select maximizes the coverage of zones and urgency but requires more time to complete the deliveries, while the priority queue delivers aid faster but has the potential to leave some zones unserved. These results work as anticipated.

Real-time Re-planning: In this test, we simulated the occurrence of road closure due to these war conflicts by blocking zone A from receiving aid. The greedy algorithm recomputes the assignments based on the remaining four zones, receiving 100% coverage on the four zones they could reach, but 76.5% when taking into account zone A, which is now unavailable to be reached. Though a route was blocked, the algorithm still took the same amount of time for deliveries to finish due to the fact that the total distance traveled remained the same (the route to zone D had to travel farther than compared to when zone A was present). This shows how blocking a high-priority route doesn't always mean that the delivery time will decrease:

Realtime re-planning example

Scenario: Zone A becomes inaccessible due to security incident

Remaining zones after blocking zone A:

Affected zones Sorted by Priority:

Zone	Demand (kg)	Urgency (1-10)	Risk (1-5)	Distance (km)	Priority Score
----- ----- ----- ----- ----- -----					
E	10000	9	5	157.0	1.273
B	8000	8	4	314.0	0.707
C	15000	7	3	470.9	0.550
D	6000	6	4	627.9	0.265

Truck Zone Assignment reasoning

Truck 1:

- Zone E: priority score 1.273, demand 10000 kg
Fit within remaining capacity (18000 kg)
- Zone B: priority score 0.707, demand 8000 kg
Fit within remaining capacity (8000 kg)
Remaining capacity after loading: 0 kg

Truck 2:

- Zone C: priority score 0.550, demand 15000 kg
Fit within remaining capacity (18000 kg)
Remaining capacity after loading: 3000 kg

Truck 3:

- Zone D: priority score 0.265, demand 6000 kg
Fit within remaining capacity (18000 kg)
Remaining capacity after loading: 12000 kg

Truck Route details -

Truck 1: assigned zones ['E', 'B']

Route order: start at depot

1. Go to Zone E - 157.0 km, arrival total 157.0 km

2. Go to Zone B - 470.9 km, arrival total 627.9 km

Then return to depot - 314.0 km, total route 941.9 km

Total travel distance: 941.9 km, estimated time: 18.8 hours (at 50 km/h)

Truck 2: assigned zones ['C']

Route order: start at depot

1. Go to Zone C - 470.9 km, arrival total 470.9 km

Then return to depot - 470.9 km, total route 941.9 km

Total travel distance: 941.9 km, estimated time: 18.8 hours (at 50 km/h)

Truck 3: assigned zones ['D']

Route order: start at depot

1. Go to Zone D - 627.9 km, arrival total 627.9 km

Then return to depot - 627.9 km, total route 1255.8 km

Total travel distance: 1255.8 km, estimated time: 25.1 hours (at 50 km/h)

New summary

Total aid delivered: 39000 kg (39.0 metric tons)

Requested aid amount (original, before blocking A): 51000 kg

Requested aid amount (after blocking A, remaining zones): 39000 kg

Coverage of remaining demand: 100.0%

Coverage of original requested aid: 76.5%

Total urgency sum: 30

Total delivery time for all trucks (combined): 62.8 hours

Urgency Decay: To verify that unserved zones become more critical over time, we advanced the simulation by 24 hours on two test zones: zone 1 (already served, urgency 5) and zone 2 (not served, urgency 2). The urgency decay formula increases unserved zones by 0.3 points per hour, capped at 10. After the time step, zone 1 remained at 5 (served zones are unchanged), while zone 2 increased to $2 + (24 * 0.3) = 9.2$. The console output confirmed these values, proving that the system correctly models worsening conditions for waiting zones:

```
Urgency Decay verification
Zone Z1: urgency = 5
Zone Z2: urgency = 9.2
```

Fog of War: Some zones are initially hidden to simulate incomplete information/intel of the zone. We tested the fog-of-war reveal function with a fixed random seed on three various hidden zones (X, Y, Z), where each zone has a 65% chance of being accessible. The function returned zone X as accessible (True), and zones Y and Z as inaccessible (False). Inaccessible fog zones within the actual software are automatically added to the blocked set and excluded from future optimization runs. These results matched our expectations, which confirms that the fog-of-war idea works as intended:

```
Fog of War verification
Zone X: accessible = True
Zone Y: accessible = False
Zone Z: accessible = False
```

Execution Results & Analysis

Execution: When running the full system using the Aleppo Siege scenario through the web interface, we configured it as such:

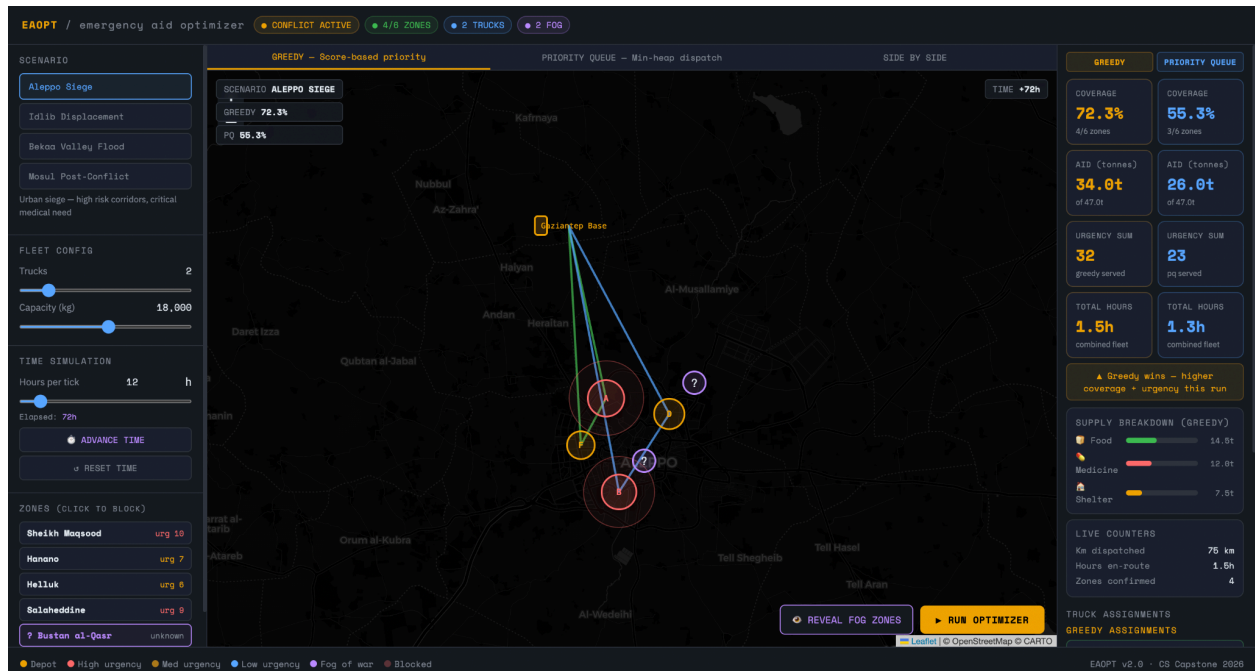
- Fleet: 2 trucks, each with 18,000 kg capacity (total fleet capacity = 36,000 kg)
- Total demand: 47,000 kg (47.0 tonnes)
- Fog zones: 2 initially hidden (Bustan al-Qasr and Masaken Hanano)
- Elapsed time: 0 hours (no urgency decay applied)

After clicking the ‘run optimizer’ button, the right panel will display the side-by-side metrics of both algorithms (greedy and priority queue). The map shows the depot (gold rectangle), the color-coded zones (red = high, orange = medium, blue = low), fog of war icons (purple with a ‘?’), and color-coded route lines from the greedy algorithm.

Results:

Algorithm	Coverage	Aid Delivered	Urgency Sum	Time Elapsed
Greedy Select	72.3%	34.0 tonnes	32	1.5 hrs
Priority Queue	55.3%	26.0 tonnes	23	1.3 hrs

The label under both algorithm results on the right panel showed that for this aid optimization, the greedy algorithm won with a higher coverage of zones and a higher urgency score.



Analysis: With an example under these constraints (36k kg capacity and demand of 47k kg), it is clear that not every zone will be able to get aid on this run. The greedy algorithm selected a combination of zones that allowed it to deliver 34,000 kilograms of aid, covering 72.3% of the zones that required materials. The priority queue only allowed the optimizer a route that would cover 55.3% of zones and deliver 26,000 kilograms of aid, but a slightly faster time elapsed of 1.3 hours compared to 1.5. This drop in performance for this situation is attributed to this algorithm's functionality of assigning zones to trucks that have the largest remaining capacity, which leads to less efficient packing of the trucks compared to the greedy algorithm's approach of filling trucks in order. The winner badge shows the best algorithm for this scenario, since greedy select had higher coverage and urgency sum, but a slightly longer delivery time by .2 hours. This example showcases the main tradeoffs of both algorithms: greedy prioritizes the amount of aid delivered and meeting the urgency of the zones, while priority queue finishes faster in some scenarios, but often doesn't meet as many of the zone demands.

Conclusions

The Emergency Aid Distribution Optimizer has demonstrated how algorithmic decision-making can help support and address the needs of areas with limited resources that are under conflict. All of the stated objectives we wanted to address were solved within this project:

1. **Two algorithms for comparison** – Greedy selection and Priority queue selection, both integrated with the nearest-neighbour routing algorithm, allow users to see the main trade-offs between coverage, urgency, and fleet hours. The side-by-side metrics panel makes this comparison immediate for users of the web interface needing help with aid distribution to areas under conflict.
2. **Realistic scenarios** – Four geographically accurate crisis zones (Aleppo Siege, Idlib Displacement, Bekaa Valley Flood, Mosul Post-Conflict) provide relevant data for our project. Each scenario uses real depot locations, zone coordinates, demand, urgency, risk, and supply types.
3. **Dynamic features** – Fog of war (hidden zones with randomised accessibility), zone blocking (manual road closures), and time-based urgency decay, make the tool responsive to the many changing conditions and uncertainty in these areas under conflict. These features directly model the uncertainty and time-sensitivity of real conflict environments.
4. **Professional user interface** – Our responsive dashboard uses Leaflet for interactive mapping, live metrics panels, an activity log, and a winner badge between both algorithms. This project design is suitable for field use on laptops or mobile devices.

AI Usage Statement

For this project, we only utilized AI to debug any errors when running the code that we wrote, for guidance on how to tackle portions of the project requirements that were confusing (used as a guide on how to solve, but not to generate code directly for us), as well as for assisting with specific technical tasks such as setting up the Flask backend structure, and converting Euclidean distances from degrees to kilometers using the appropriate conversion factor. All of the logic for this project, including algorithm design (greedy selection, priority queue, nearest-neighbour routing, urgency decay, fog of war), data structuring for the four scenarios, front-end UI implementation, and overall system integration, were written by my teammate and me.

References

ACAPS. ACAPS – Independent Humanitarian Analysis. www.acaps.org/en/.

Provides the independent crisis analysis and humanitarian needs assessments that we used to design our four real-world scenarios for this aid optimizer (Aleppo, Idlib, Bekaa, and Mosul).

GeeksforGeeks. “K-Nearest Neighbours.” GeeksforGeeks, www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/.

This article explains the K-nearest neighbour algorithm that helped influence our nearest neighbour routing algorithm.

GeeksforGeeks. “Greedy Algorithms.” GeeksforGeeks, www.geeksforgeeks.org/dsa/greedy-algorithms/.

This resource describes the principles of the greedy algorithm, which is used as a reference for the zone prioritization and truck allocation algorithm (greedy_select).

United Nations Office for the Coordination of Humanitarian Affairs (UN OCHA). UN OCHA. www.unocha.org/.

This source detailing real-world humanitarian reports on many locations conflicted with war provided much contextual information in order for us to build this project and its scenarios.